

Generador de señales programable

Branca, Gonzalo

Condori, Ricardo

Abstract—Este documento describe el diseño e implementación de un generador de señales programable basado en el microcontrolador ESP32 y el sistema operativo en tiempo real FreeRTOS . El sistema permite generar señales periódicas (senoidales, cuadradas y triangulares) con una frecuencia configurable entre 1 Hz y 20 kHz, una amplitud de 100 mVpp a 5 Vpp, y un nivel de offset ajustable de $\pm 2,5$ V . La arquitectura de hardware incorpora una interfaz hombre-máquina (HMI) compuesta por encoders, potenciómetros y una pantalla OLED vía I2C, sumado a un bus UART para su configuración remota . La señal es sintetizada mediante un DAC externo de 12 bits por SPI, seguido de etapas analógicas de reconstrucción, amplificación y protección contra cortocircuitos . A nivel de firmware, el procesamiento se modulariza en cuatro tareas concurrentes de prioridades escalonadas, sincronizadas mediante colas de datos y semáforos . Finalmente, para garantizar la temporización determinista exigida, la actualización de las muestras en el DAC se ejecuta mediante interrupciones de hardware (DMA) independientes del planificador del sistema operativo

Index Terms—component, formatting, style, styling, insert

I. INTRODUCCION Y ALCANCE

En el presente documento se describe el diseño, la fundamentación teórica y la arquitectura del proyecto correspondiente a la Consigna 5: "Generador de señales programable" . El objetivo principal es desarrollar un instrumento de laboratorio basado en el microcontrolador ESP32 que, aprovechando la concurrencia y los recursos del sistema operativo en tiempo real FreeRTOS , sea capaz de generar señales periódicas configurables de manera local y remota. Como alcance propuesto para la solución, el sistema sintetizará tres tipos de formas de onda: senoidales, cuadradas y triangulares . Los parámetros de estas señales podrán ser modificados en tiempo real por el usuario, garantizando un rango de frecuencia configurable entre 1 Hz y 20 kHz (con una resolución mínima de 1 Hz), una amplitud pico a pico ajustable entre 100 mVpp y 5 Vpp, y un nivel de tensión de offset regulable en un rango de $\pm 2,5$ V . Para cumplir con estos requerimientos, la arquitectura de hardware propuesta delegará la generación analógica a un convertor digital-analógico (DAC) externo de 12 bits comunicado mediante el bus SPI, seguido de una etapa de reconstrucción de señal mediante filtrado y una etapa de amplificación que incorporará protección por hardware contra cortocircuitos a tierra en la salida . A nivel de software, el firmware se estructurará de forma modular dividiendo las responsabilidades lógicas en tareas de diferentes prioridades gestionadas por FreeRTOS. Asimismo, para asegurar que la señal generada no sufra deformaciones temporales, la escritura de cada muestra en el DAC se realizará de forma determinista utilizando rutinas de interrupción de hardware (DMA),

operando con total independencia del planificador de tareas del sistema operativo .

II. ARQUITECTURA DE HARDWARE

La arquitectura de hardware del generador de señales se diseñó de manera modular, centralizando el procesamiento en un microcontrolador ESP32 y delegando la síntesis analógica y la interacción con el usuario a periféricos externos específicos.

A. Diagrama en Bloques

En la Figura 1 se detalla el diagrama en bloques del sistema, evidenciando las interconexiones físicas y los pines GPIO asignados a cada módulo.

B. Especificaciones y Funcionamiento de Módulos

1) Interfaz Hombre-Máquina (HMI): Para la configuración local, el sistema incorpora un Encoder Rotativo KY-040 conectado a los pines GPIO 5, 18 y 19, el cual permite ajustar la frecuencia y cambiar la escala de ajuste. Adicionalmente, se utilizan potenciómetros conectados a los canales del ADC1 para el ajuste fino de la amplitud (100 mVpp a 5 Vpp) y el nivel de offset ($\pm 2,5$ V). La retroalimentación visual se realiza mediante una pantalla OLED de 0.96 pulgadas conectada a través del bus I2C utilizando los pines GPIO 21 (SDA) y GPIO 22 (SCL).

2) Comunicación Remota: Se implementó un bus de comandos externos gestionado mediante la interfaz UART2 del ESP32 (TXD en GPIO 17 y RXD en GPIO 16), permitiendo la configuración remota del instrumento en cumplimiento con los lineamientos de estandarización del proyecto.

3) Generación de Señal (DAC Externo): Para cumplir con la resolución exigida y evitar las limitaciones del hardware interno del microcontrolador, se integró un convertor digital-analógico (DAC) externo de 12 bits. Este módulo se comunica a alta velocidad mediante el bus SPI por hardware, utilizando los pines GPIO 13 (MOSI), GPIO 14 (CLK) y GPIO 15 (CS).

C. Esquemáticos y Acondicionamiento Analógico

La salida del DAC externo requiere de un acondicionamiento analógico para cumplir con las tolerancias eléctricas del proyecto. A continuación se presentan los esquemáticos desarrollados para esta etapa.

Justificación de Componentes: A la salida del DAC, se implementó un filtro pasa bajos de reconstrucción. El objetivo de este componente es atenuar las componentes de alta frecuencia producto del muestreo digital, suavizando los escalones de tensión para obtener señales continuas y limpias en todo el rango de operación (1 Hz a 20 kHz). Posteriormente,

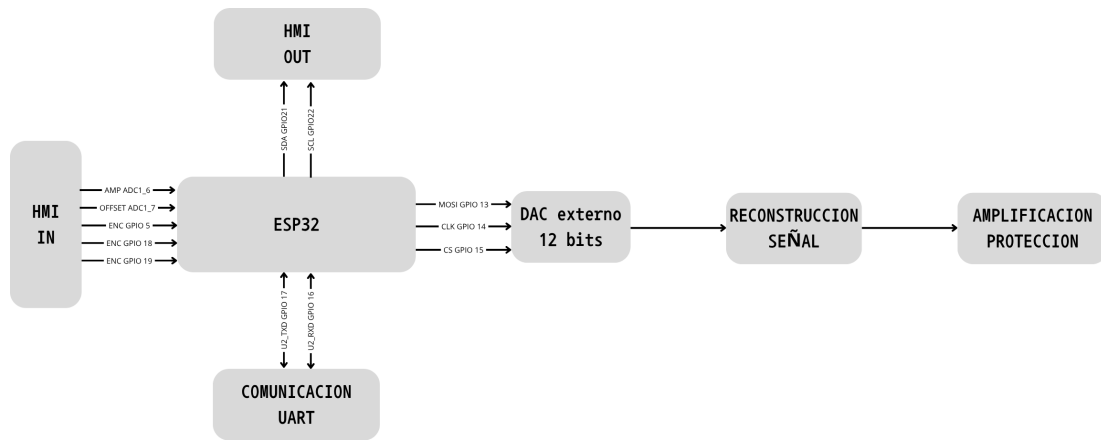


Fig. 1. Diagrama en bloques del sistema indicando interconexión de pines.

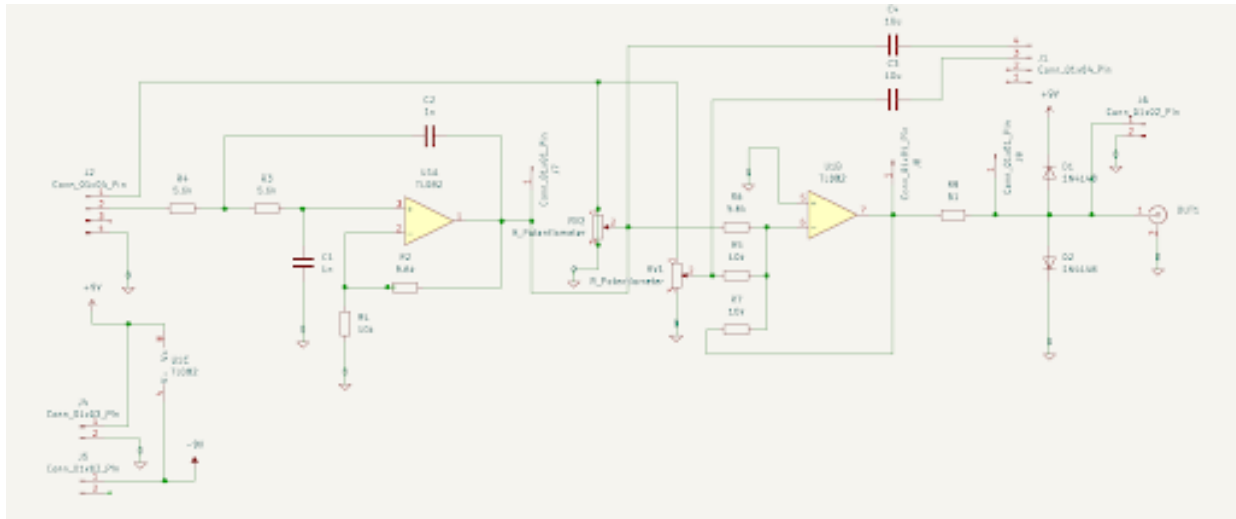


Fig. 2. Esquemático de la etapa amplificación y salida.

la señal ingresa a una etapa de amplificación y adaptación de niveles, garantizando excursiones de hasta 5 Vpp. Finalmente, se diseñó una etapa de protección por hardware en la salida que limita la corriente máxima, asegurando que el instrumento tolere una conexión directa a tierra (cortocircuito) sin sufrir daños en sus componentes.

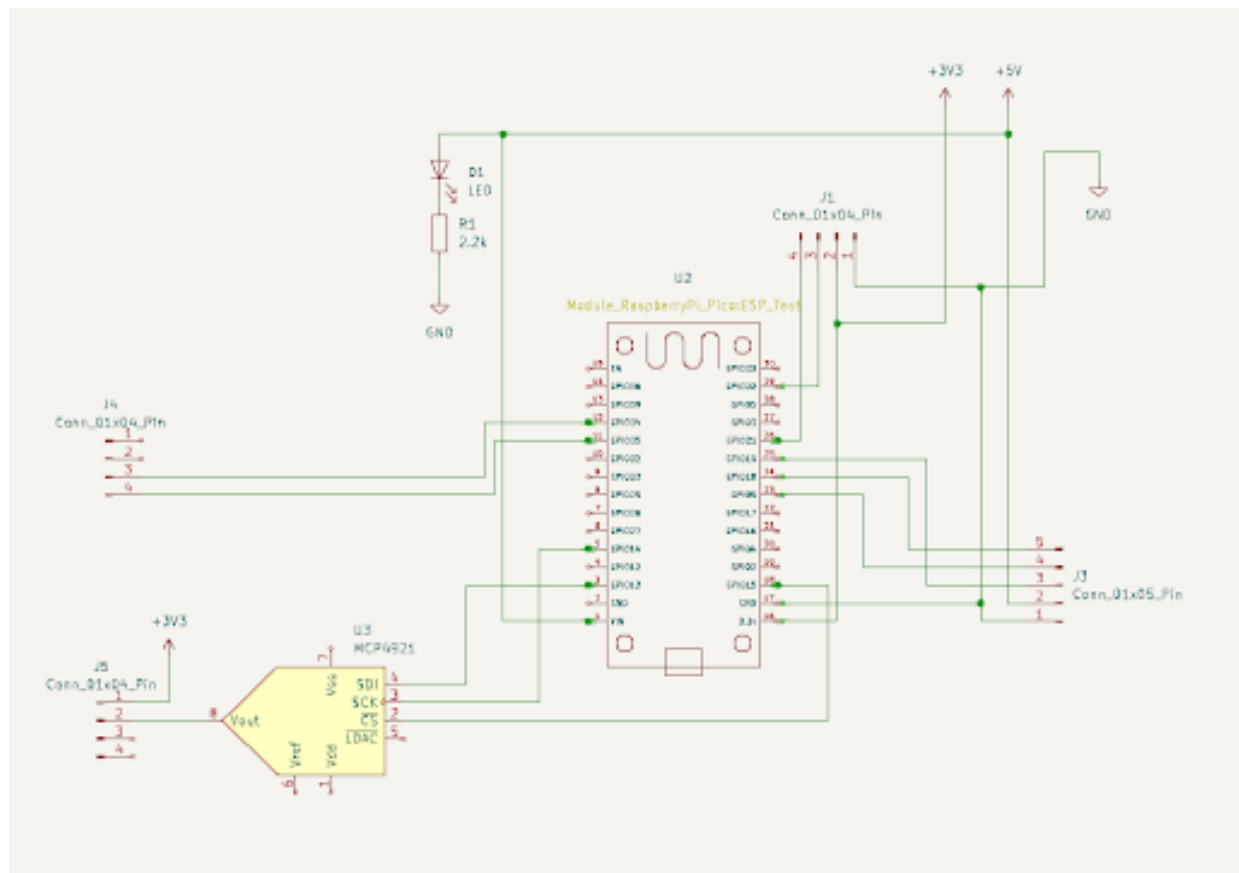


Fig. 3. Esquemático de la etapa de control.

III. ARQUITECTURA DE SOFTWARE Y GUÍA DE CÓDIGO

El firmware del dispositivo fue diseñado bajo el paradigma de un Sistema Operativo en Tiempo Real, utilizando FreeRTOS para garantizar la concurrencia, modularidad y el cumplimiento de las restricciones temporales del sistema. A continuación, se detalla la guía de código mediante el diagrama de tareas propuesto.

A. Relación entre Funciones, Tareas y Recursos

Tal como se observa en la Figura 4, la arquitectura de software se divide en cuatro tareas con prioridades escalonadas y dos rutinas de interrupción de hardware (ISR), las cuales se comunican mediante primitivas del sistema operativo:

1) Tarea_HMI (Prioridad 1 - Baja): Encargada de leer de forma periódica los periféricos de entrada locales (el encoder rotativo y los potenciómetros) y actualizar la interfaz gráfica en la pantalla OLED. Al detectar un cambio, empaqueta los parámetros actualizados y actúa como una tarea escritora, enviándolos a la Cola de Datos.

2) Tarea_comandos (Prioridad 2 - Media): Gestiona la configuración remota mediante el bus UART. Esta tarea permanece en estado bloqueado para ahorrar tiempo de procesador. Se despierta únicamente cuando la interrupción `ISR_UART` recibe un byte físico de hardware y libera un Semáforo de sincronización. Al despertar, decodifica la trama entrante y envía los nuevos parámetros a la Cola de Datos.

3) Tarea_procesamiento (Prioridad 3 - Alta): Funciona como la única tarea lectora de la Cola de Datos. Al recibir una nueva configuración (ya sea de la HMI o de la UART), calcula la *Look-Up Table* (LUT) de la forma de onda deseada, ajusta la frecuencia y guarda el estado actual en la memoria no volátil para garantizar la persistencia de datos ante un reinicio [2].

4) Tarea_watchdog (Prioridad 4 - Máxima): Implementa el mecanismo obligatorio de recuperación ante fallos. Monitorea periódicamente la correcta ejecución del resto de las tareas y fuerza un reinicio del microcontrolador en caso de detectar bloqueos o puntos muertos en la ejecución.

5) Determinismo e Interrupciones (ISR_DMA): Para garantizar que la generación analógica cumpla con la temporización determinista estricta y pueda alcanzar frecuencias de 20 kHz sin fluctuaciones, el envío de las muestras de la LUT hacia el DAC se realiza operando con total independencia del *scheduler* de FreeRTOS. Esto se logra mediante una interrupción de hardware y transferencia por DMA (`ISR_DMA`).

IV. DISEÑO DE PCB Y PUNTOS DE PRUEBA

Como paso previo al ruteo de la placa de circuito impreso (PCB) requerida para la próxima instancia de evaluación, se establecieron los puntos de prueba (*test points*) estratégicos del sistema. Estos permitirán verificar y diagnosticar el hardware con instrumental de laboratorio durante el ensamblaje:

- **TP1 (3.3V) y TP2 (GND):** Medición de la alimentación lógica general del microcontrolador y los periféricos.

- **TP3 (DAC_OUT):** Salida directa del convertor digital-analógico para verificar la integridad de los datos recibidos por el bus SPI.
- **TP4 (FILTRO_OUT):** Salida de la etapa de reconstrucción analógica, utilizada para validar la correcta atenuación de las componentes de alta frecuencia.
- **TP5 (AMP_OUT):** Salida final del generador, indispensable para corroborar la excursión máxima de amplitud (hasta 5 Vpp) y el barrido del offset ($\pm 2,5$ V).
- **TP6 (UART_TX) y TP7 (UART_RX):** Puntos de monitoreo para depurar las tramas de la comunicación remota.

V. CONCLUSIONES PRELIMINARES

A nivel de anteproyecto, la arquitectura conjunta de hardware y firmware propuesta resuelve de manera integral los requerimientos técnicos de la consigna. La implementación de FreeRTOS demostró ser fundamental para desacoplar las tareas críticas de temporización, delegando la carga del DAC a interrupciones de hardware (DMA), lo que previene la fluctuación temporal (*jitter*) en la señal generada.

REFERENCES

- [1] Espressif Systems, *ESP32 Series Datasheet*, 2023. [Online]. Disponible en: <https://www.espressif.com/>
- [2] Real Time Engineers Ltd., *FreeRTOS API Reference Manual*. [Online]. Disponible en: <https://www.freertos.org/>
- [3] Microchip Technology Inc., *MCP4921/4922 12-Bit Digital-to-Analog Converter with SPI Interface Datasheet*.

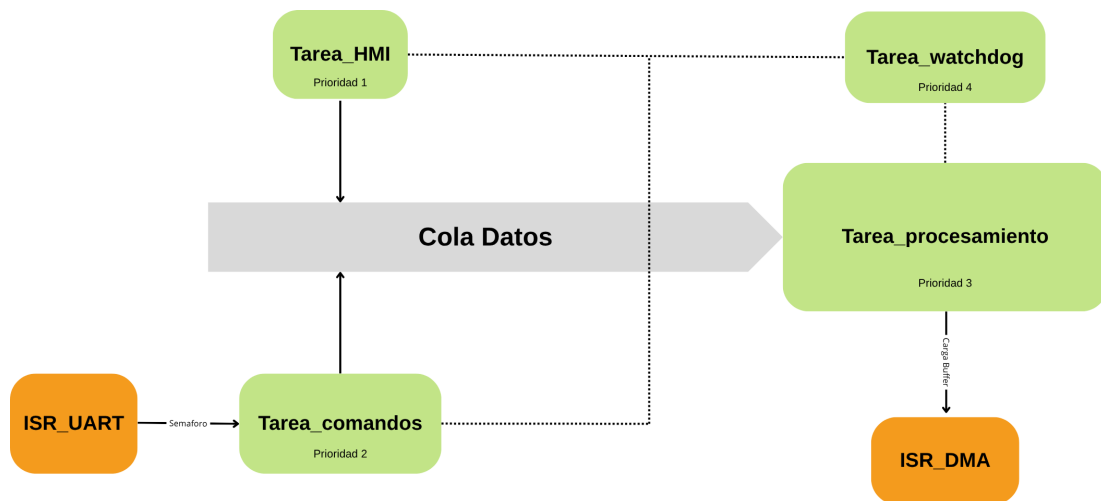


Fig. 4. Diagrama de tareas evidenciando responsabilidades y recursos de FreeRTOS.